Lab 04 · Canary de Lambda con rollback automático

Duración estimada: 50 minutos **Sesión:** 2 **Entorno:** AWS CloudShell y consola · región `eu-west-1` (Irlanda)

Objetivo

Desplegar una función Lambda con AWS SAM y AWS CodeDeploy usando una estrategia *canary* (10 % del tráfico durante 5 minutos) protegida por una alarma de CloudWatch. Primero publicarás una versión sana y verás cómo el canary termina con éxito; después publicarás una versión defectuosa mientras generas tráfico y comprobarás que el despliegue se revierte solo, sin intervención manual. Terminarás analizando métricas, historial de la alarma y logs como harías en un *postmortem*.

Requisitos

- Permisos sobre CloudFormation, IAM, Lambda, CodeDeploy, CloudWatch y CloudWatch Logs en la cuenta de formación.
- Dos pestañas de CloudShell abiertas (en CloudShell: **Actions → New tab**).

Punto de partida

La práctica es independiente: todo lo que necesitas se crea en el paso 1 desde una única plantilla. El código de la función va **en línea** dentro de la plantilla (`InlineCode`), de modo que no hace falta compilar ni empaquetar nada y basta con la AWS CLI que ya trae CloudShell.

> **Cómo funciona por dentro.** La propiedad `AutoPublishAlias` hace que SAM publique una versión nueva de la función cada vez que cambia su código y que mantenga un alias `live` apuntando a la versión en servicio. La propiedad `DeploymentPreference` hace que ese cambio de alias no sea instantáneo: SAM crea una aplicación y un grupo de despliegue de CodeDeploy, y es CodeDeploy quien mueve el tráfico del alias poco a poco mientras vigila la alarma. Si la alarma salta, CodeDeploy devuelve todo el tráfico a la versión anterior y CloudFormation marca la actualización como fallida.

Paso 1 · Preparar la plantilla y desplegar la versión 1

En la **pestaña 1** de CloudShell:

```
export SUFIJO=jlb01            # tu sufijo, en minúsculas
export AWS_REGION=eu-west-1
export STACK=lab04-canary-$SUFIJO
mkdir -p ~/lab04 && cd ~/lab04

cat > plantilla.yaml <<'PLANTILLA'
AWSTemplateFormatVersion: "2010-09-09"
Transform: AWS::Serverless-2016-10-31
Description: Lab 04 - canary de Lambda con rollback automatico
Parameters:
  Sufijo:
    Type: String
Resources:
  Funcion:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: !Sub "lab04-fn-${Sufijo}"
      Runtime: python3.12
      Handler: index.handler
      Timeout: 5
      MemorySize: 128
      AutoPublishAlias: live
      DeploymentPreference:
        Type: Canary10Percent5Minutes
        Alarms:
          - !Ref AlarmaErrores
```

```
      InlineCode: |
        import json, random
        VERSION = "1.0.0"
        TASA_ERROR = 0.0
        def handler(event, context):
            if random.random() < TASA_ERROR:
                raise RuntimeError("Fallo simulado en la version " + VERSION)
            return {"statusCode": 200, "version": VERSION}

  AlarmaErrores:
    Type: AWS::CloudWatch::Alarm
    Properties:
      AlarmName: !Sub "lab04-errores-nueva-version-${Sufijo}"
      AlarmDescription: Errores en la version nueva durante el despliegue
      Namespace: AWS/Lambda
      MetricName: Errors
      Dimensions:
        - Name: Resource
          Value: !Sub "${Funcion}:live"
        - Name: FunctionName
          Value: !Ref Funcion
        - Name: ExecutedVersion
          Value: !GetAtt Funcion.Version.Version
      Statistic: Sum
      Period: 60
      EvaluationPeriods: 1
      Threshold: 0
      ComparisonOperator: GreaterThanThreshold
      TreatMissingData: notBreaching

Outputs:
  NombreFuncion:
    Value: !Ref Funcion
  VersionPublicada:
    Value: !GetAtt Funcion.Version.Version
PLANTILLA
```

Fíjate en tres detalles de la alarma, porque son los que suelen fallar en proyectos reales. La dimensión `ExecutedVersion` apunta a la versión **recién publicada**, así que la alarma solo mide los errores de la versión nueva y no los de la antigua. `TreatMissingData: notBreaching` evita que la falta de datos (por ejemplo, cuando no hay tráfico) se interprete como alarma. Y el umbral `> 0` es deliberadamente estricto para que el laboratorio sea rápido; en producción usarías una tasa de error o un percentil de latencia.

Despliega:

```
aws cloudformation deploy \
  --stack-name $STACK \
  --template-file plantilla.yaml \
  --parameter-overrides Sufijo=$SUFIJO \
  --capabilities CAPABILITY_IAM CAPABILITY_AUTO_EXPAND
```

`CAPABILITY_AUTO_EXPAND` es necesario porque la plantilla usa la transformación de SAM, y `CAPABILITY_IAM` porque SAM crea el rol de ejecución de la función y el rol de servicio de CodeDeploy.

> **Punto de control 1.** El comando termina con `Successfully created/updated stack`. En la primera creación no hay canary: no existe una versión anterior a la que desplazar tráfico, así que el alias `live` nace apuntando directamente a la versión 1.

Comprueba el alias y haz una invocación:

```
aws lambda get-alias --function-name lab04-fn-$SUFIJO --name live
aws lambda invoke --function-name lab04-fn-$SUFIJO:live --payload '{}' \
  --cli-binary-format raw-in-base64-out /dev/stdout
```

Debes ver `"FunctionVersion": "1"` y la respuesta `{"statusCode": 200, "version": "1.0.0"}`.

Paso 2 · Generar tráfico continuo

Pasa a la **pestaña 2** de CloudShell y lanza un bucle que invoca el alias una vez por segundo. Muestra la hora, la versión que responde o el mensaje de error:

```
export SUFIJO=jlb01            # el mismo sufijo que en la pestaña 1
export AWS_REGION=eu-west-1
while true; do
  aws lambda invoke --function-name lab04-fn-$SUFIJO:live --payload '{}' \
    --cli-binary-format raw-in-base64-out /tmp/respuesta.json > /tmp/meta.json
  printf "%s  " "$(date +%H:%M:%S)"
  if grep -q FunctionError /tmp/meta.json; then
    echo "ERROR -> $(cat /tmp/respuesta.json | head -c 120)"
  else
    cat /tmp/respuesta.json; echo
  fi
  sleep 1
done
```

Déjalo corriendo durante el resto de la práctica. Por ahora todas las líneas deben mostrar la versión `1.0.0`.

> **Por qué hace falta tráfico.** Un canary solo protege si la versión nueva recibe peticiones durante la ventana de observación. Sin tráfico no hay errores, la alarma no salta y el despliegue termina «con éxito» aunque el código esté roto. En producción, donde el tráfico nocturno puede ser mínimo, esto se resuelve con pruebas sintéticas (por ejemplo, CloudWatch Synthetics) o con un *hook* `PreTraffic` que invoca la versión nueva antes de desplazar tráfico.

Paso 3 · Publicar una versión sana (1.1.0) y ver un canary que termina bien

Vuelve a la **pestaña 1**. Genera una versión nueva cambiando solo el número de versión y despliega:

```
sed -e 's/VERSION = "1.0.0"/VERSION = "1.1.0"/' plantilla.yaml > plantilla-v11.yaml
aws cloudformation deploy \
  --stack-name $STACK \
  --template-file plantilla-v11.yaml \
  --parameter-overrides Sufijo=$SUFIJO \
  --capabilities CAPABILITY_IAM CAPABILITY_AUTO_EXPAND
```

El comando se quedará esperando unos seis o siete minutos: CloudFormation no da por terminada la actualización hasta que CodeDeploy completa el despliegue. Mientras tanto:

1. En la **pestaña 2**, observa que aproximadamente una de cada diez respuestas ya dice `1.1.0`.
2. Abre la consola de **CodeDeploy → Deployments**. Verás un despliegue en curso de la aplicación `lab04-canary-<sufijo>-ServerlessDeploymentApplication-...`. Entra en él y observa el reparto de tráfico entre la versión original y la de reemplazo.
3. Desde otra ventana del navegador o abriendo una pestaña 3 de CloudShell, consulta el alias durante el canary:

```
aws lambda get-alias --function-name lab04-fn-$SUFIJO --name live \
  --query "{principal:FunctionVersion, pesos:RoutingConfig.AdditionalVersionWeights}"
```

Verás que el alias sigue apuntando a la versión 1 como principal, pero con un peso de `0.1` hacia la versión 2. Así es como Lambda implementa el canary: un alias con enrutamiento ponderado entre dos versiones.

> **Punto de control 2.** Pasados los cinco minutos, CodeDeploy mueve el 100 % del tráfico, el despliegue queda en estado **Succeeded**, el comando de la pestaña 1 termina correctamente y en la pestaña 2 todas las respuestas pasan a `1.1.0`.

Paso 4 · Publicar una versión defectuosa (2.0.0) y ver el rollback

Ahora la versión nueva falla en la mitad de las invocaciones. En la **pestaña 1**:

```
sed -e 's/VERSION = "1.0.0"/VERSION = "2.0.0"/' \
    -e 's/TASA_ERROR = 0.0/TASA_ERROR = 0.5/' plantilla.yaml > plantilla-v2.yaml
grep -E "VERSION =|TASA_ERROR =" plantilla-v2.yaml
aws cloudformation deploy \
  --stack-name $STACK \
  --template-file plantilla-v2.yaml \
  --parameter-overrides Sufijo=$SUFIJO \
  --capabilities CAPABILITY_IAM CAPABILITY_AUTO_EXPAND
```

Mientras el comando espera, sigue lo que ocurre:

1. En la **pestaña 2** empiezan a aparecer, de vez en cuando, líneas `ERROR -> ... Fallo simulado en la version 2.0.0`. Fíjate en que son pocas: solo el 10 % del tráfico va a la versión 2 y, de ese tráfico, falla la mitad. Es exactamente la idea del canary: el daño queda acotado a una fracción pequeña de usuarios.
2. En **CloudWatch → Alarms**, la alarma `lab04-errores-nueva-version-<sufijo>` pasa de **OK** a **In alarm** en uno o dos minutos.
3. En **CodeDeploy → Deployments**, el despliegue pasa a **Stopped** o **Failed** con el motivo de que se ha activado una alarma, y CodeDeploy devuelve todo el tráfico a la versión 1.1.0.
4. En la **pestaña 1**, el comando termina con un error: la actualización del stack ha fallado y CloudFormation hace rollback a la configuración anterior.

> **Punto de control 3.** En la pestaña 2 todas las respuestas vuelven a ser `1.1.0` sin errores. Compruébalo también con el alias:

```
aws lambda get-alias --function-name lab04-fn-$SUFIJO --name live \
  --query "{principal:FunctionVersion, pesos:RoutingConfig.AdditionalVersionWeights}"
aws cloudformation describe-stacks --stack-name $STACK \
  --query "Stacks[0].StackStatus" --output text
```

El alias apunta de nuevo a la versión 2 (la que contiene `1.1.0`) sin pesos adicionales, y el stack queda en `UPDATE_ROLLBACK_COMPLETE`, que es un estado estable desde el que se puede volver a desplegar. La versión 3 de la función (la defectuosa) sigue existiendo, pero no recibe tráfico.

Ya puedes detener el bucle de la pestaña 2 con `Ctrl+C`.

Paso 5 · Analizar lo ocurrido con métricas y logs

Un rollback automático resuelve la urgencia, pero el equipo necesita entender qué pasó. Reconstruye la historia con tres fuentes.

Historial de la alarma, que te da la hora exacta en que se detectó el problema:

```
aws cloudwatch describe-alarm-history \
  --alarm-name lab04-errores-nueva-version-$SUFIJO \
  --history-item-type StateUpdate \
  --query "AlarmHistoryItems[].[Timestamp,HistorySummary]" --output table
```

Errores por versión ejecutada, para confirmar que solo falló la versión nueva. En la consola de **CloudWatch → Metrics → All metrics → Lambda → By Function Name, Resource and Executed Version** (o similar, según cómo la agrupe la consola), marca la métrica `Errors` de tu función para las versiones 2 y 3 y usa la estadística *Sum* con periodo de 1 minuto. Verás errores solo en la versión 3 y únicamente durante la ventana del canary.

Logs de la función, para ver el error concreto y su traza:

```
aws logs tail /aws/lambda/lab04-fn-$SUFIJO --since 30m --filter-pattern "Fallo simulado"
```

Con estas tres piezas escribe, en cuatro o cinco líneas, un mini *postmortem*: qué versión se desplegó y a qué hora, cuándo saltó la alarma, cuánto tráfico se vio afectado (aproximadamente) y cuánto tardó en restablecerse el servicio. Lo comentaremos en común.

Reto opcional

Cambia `Canary10Percent5Minutes` por `Linear10PercentEvery1Minute`, publica de nuevo una versión defectuosa (por ejemplo, `2.1.0` con `TASA_ERROR = 0.5`) y compara cuánto tráfico llega a la versión nueva antes de que la alarma la detenga. Si quieres ir más allá, prueba `AllAtOnce` con la misma versión defectuosa y observa que, aunque la alarma sigue provocando el rollback, durante ese intervalo el 100 % del tráfico sufre los errores.

Preguntas de reflexión

1. ¿Qué habría pasado si no hubieras tenido el bucle de tráfico en marcha durante el despliegue de la versión 2.0.0?
2. ¿Por qué la alarma filtra por `ExecutedVersion` en lugar de medir los errores de toda la función?
3. En un servicio con un 0,1 % de errores normales, ¿cómo redefinirías la alarma para que un canary no se revierta por el ruido de fondo?

Limpieza

Asegúrate de haber detenido el bucle de la pestaña 2 y ejecuta en la pestaña 1:

```
aws cloudformation delete-stack --stack-name $STACK
aws cloudformation wait stack-delete-complete --stack-name $STACK
aws logs delete-log-group --log-group-name /aws/lambda/lab04-fn-$SUFIJO
```

El borrado del stack elimina la función con todas sus versiones, el alias, la alarma, la aplicación y el grupo de despliegue de CodeDeploy y los roles que creó SAM. El grupo de logs lo crea Lambda al ejecutarse, fuera del stack, por eso se borra aparte.